

Nolte Geoservices UXO Target Platform

ETL Operator, Maintenance & Scheduling Architecture Guide

Document Version: 2.4	Target Environment: Production / Staging	Last Updated: 2026-09-30	Status: Fully Operational
-----------------------	--	--------------------------	---------------------------

Overview

The ETL pipeline coordinates data between source geodata tables (managed in QGIS/GIS workflows) and the live production application (`public.anomalies`, `public.feedback`).

This guide covers operational procedures, configuration validation, Web UI approvals, project-scoped runs, webhook notifications, and Slowly Changing Dimensions (SCD Type 2) audit history tracking.

1. Quick Reference & Core Commands

Action	Command / Location	Privileges Required
Trigger Full Run	Web UI: Database icon -> Run Sync Now CLI: <code>docker compose --profile etl run --rm etl run</code> (add <code>--force</code> to bypass the nothing-changed check)	Admin user (UI) / <code>etl_pipeline</code> (CLI)
Trigger Single Project	Web UI: Select project -> Run Sync Now CLI: <code>docker compose --profile etl run --rm etl run --project 11-26-5151</code>	Admin user (UI) / <code>etl_pipeline</code> (CLI)
Start Cron Orchestrator	CLI: <code>docker compose --profile etl run --rm etl cron --schedule "/15 *"</code> Daemon: Container running with <code>ETL_CRON_SCHEDULE</code>	<code>etl_pipeline</code>
Validate Configuration	<code>docker compose --profile etl run --rm etl validate-config</code>	Read-only
Check Pipeline Health	Web UI: Top KPI Ribbon API: <code>GET /api/etl/health</code> (public, status only) <code>GET /api/etl/health/details</code> (admin)	Read-only
Review Approvals (Web)	Web UI: Click Database icon in navigation rail	Admin user (<code>etl_approver</code>)
Review Approvals (CLI)	<code>docker compose --profile etl run --rm etl-approve status --approver</code>	<code>etl_approver</code>
Approve / Reject Staged	<code>docker compose --profile etl run --rm etl-approve approve-change <ids...></code> <code>docker compose --profile etl run --rm etl-approve reject-change <ids...></code>	<code>etl_approver</code>
Approve / Reject Shift	<code>docker compose --profile etl run --rm etl-approve approve-correction <pair_id></code> <code>docker compose --profile etl run --rm etl-approve reject-correction <pair_id></code>	<code>etl_approver</code>
Inspect Audit History	Web UI: Database -> Audit History tab API: <code>GET /api/etl/history?limit=100</code>	Admin user (<code>etl_approver</code>)

2. Configuration & Validation

The pipeline configuration is stored in `etl/config/projects.yml`.

Configuration Linter

Before deploying or running after editing `projects.yml`, run the config linter:

```
docker compose --profile etl run --rm etl validate-config
```

The linter validates:

- **Global Settings:** Validates `correction_radius_m` is positive.

- **Projects:** Verifies project IDs follow the **YY-NN-NNNN** pattern, schemas exist, and SRID is a valid positive integer (e.g., 25832, 25833).
- **Sources:** Verifies table names, instruments (**magnetic**, **georadar**), and coordinate column definitions (**easting**, **northing**).
- **Data Types & Bounds:** Verifies coordinate columns in Postgres are numeric/float types and fall within valid UTM boundaries.

What a single-project run does

run --project <id> (or a project picked in the web UI) still **validates and builds every project**: dbt always rebuilds **etl.stg_candidates**, **etl.int_candidates**, **etl.excluded_rows** and **etl.dup_report** for all of them, so those tables are never left holding one project. Only these are limited to the chosen project:

- the merge into **public.anomalies** (inserts, staged changes, correction pairs);
- the approver's decisions carried out: other projects' approved or rejected decisions wait for their own run or the next full run;
- the per-project row-count gates, and the report.

Because every project is built, a broken source table in any project fails a single-project run too, exactly as it fails a full run.

3. Web Approver & Runner Interface

Admins can manage the entire ETL lifecycle from the web application without using command-line tools:

- 1. Access:** Click the **Database** icon in the left-hand navigation rail. A badge shows pending approvals in real-time.
- 2. Top Health Ribbon:** Displays real-time **Pipeline Health** (Healthy/Degraded/Unhealthy), database connection ping latency (ms), freshness SLA status, and counts of pending staged changes and coordinate shifts.
- 3. Execution Bar:**
 - **Project Selector:** Run across all projects or select a specific project (**11-26-5151**, **11-24-2736**).
 - **Force Sync Toggle** (off by default): bypasses the "nothing changed since the last successful run" check. Not needed to carry out new decisions: an open decision already counts as a change.
 - **Run Sync Now:** runs the pipeline once in a one-off ETL container and waits for it (up to 180 seconds), then refreshes the panel. It reports **completed**, **nothing to do** (skipped), or **failed**; if another run holds the pipeline's lock it does nothing and says so (HTTP 409). A run that outlasts the wait is not stopped: it appears in the runs list when it ends.
- 4. Staged Changes Tab:**
 - Shows attribute differences (**category**, **layer**, **evaluated_depth**, **instrument**).
 - Select individual rows or use "Select All" for bulk approve/reject. A bulk decision is all-or-nothing: if one change can no longer be decided (no longer staged, already decided), none of the batch is recorded and the reason is returned.
 - Every decision records who took it in **etl_approval.decisions.decided_by: etl_approver via app user <username>** from the web UI, plain **etl_approver** from the CLI.
- 5. Coordinate Corrections Tab:**
 - Compares existing coordinates with new source coordinates.
 - Displays spatial shift distance in meters and rule match rationale (**source_key** vs spatial proximity).
- 6. Runs & Health Tab:**
 - Inspect the last 25 pipeline executions with interactive SVG charts for duration trends (latency), ingestion volume vs staged changes, outcome distribution, and run status filters (**All**, **Successful**, **Skipped**, **Failed**).
- 7. Audit History (SCD Type 2) Tab:**
 - Inspect full historical timeline of all changes made to targets.
 - Search by Target ID, VM Number, Project, or Change Reason.

4. Slowly Changing Dimensions (SCD Type 2) Audit History

Target history is maintained in **public.anomaly_history** through the PostgreSQL trigger **trigger_anomaly_scd_audit**.

Lifecycle:

- 1. Creation:** When a new anomaly is inserted, an initial record is created with **is_current = true**, **valid_from = now()**, and **valid_to = NULL**.
- 2. Modifications:** When attributes or coordinates change:
 - The prior active record is closed (**valid_to = now()**, **is_current = false**).
 - A new current record is inserted (**valid_from = now()**, **valid_to = NULL**, **is_current = true**).

- The change reason is tagged automatically (`etl_change`, `etl_correction`, `field_sync`, `initial`).
- If applied via ETL approver, the associated `decision_id` is recorded.
- 3. Integrity:** A partial unique index (`anomaly_id`) `WHERE is_current = true` enforces at most one current record per target at the database engine level.
- 4. Append-only:** the trigger `trigger_anomaly_history_append_only` refuses every `DELETE`, `TRUNCATE` and `UPDATE` on `anomaly_history` except the audit trigger closing the current version. There is no foreign key to `anomalies`: deleting a target, or its project, keeps its full history. Only disabling the trigger (superuser) gets around this.

5. Webhook Notifications

Set `ETL_WEBHOOK_URL` in `.env` to enable automated alerts:

- **Slack / Teams / Discord / Generic HTTP:** Posts JSON notifications automatically.
- **Alert Events:**
 - **FAILED:** Triggered on unhandled errors or safety gate failures with traceback.
 - **PENDING:** Triggered when a run holds back changes or shifts awaiting human review.
 - **SUCCESS:** Triggered on successful merges summarizing written and staged row counts.

6. Security & Privilege Separation

- `etl_pipeline`: Runs the pipeline. Possesses `INSERT` and `SELECT` on targets, but **never** `UPDATE` or `DELETE` on `public.anomalies`. It executes changes exclusively via PostgreSQL `SECURITY DEFINER` functions in `etl_admin`.
- `etl_approver`: Used exclusively for reviewing and deciding staged changes and coordinate shifts. It cannot write directly to `public.anomalies`.
- All decisions require explicit human review before modifying production records.

7. CI/CD Automated Validation

A GitHub Actions pipeline (`.github/workflows/etl-ci.yml`) automatically validates changes on every push and pull request touching ETL or frontend components:

- **Config & Schema Linting:** Validates `projects.yml` syntax, positive integer SRIDs, and column requirements (`python runner/run.py validate-config`).
- **Automated Unit Tests:** Executes unit test suites verifying schema linter, webhook payloads, and spatial bounds.
- **Frontend Quality Gates:** Executes ESLint and compiles production Vite bundles (`npm run build`).

8. Structured JSON Logging & Rotation

The pipeline features dual-destination structured logging via `etl/runner/logger.py`:

- **Console (stdout):** Formatted, human-readable execution output tagged with run and project IDs (`[2026-09-29 11:00:00] [INFO] [run:72] [proj:11-26-5151] ...`).
- **Rotating JSON Lines File (`etl/logs/etl.jsonl`):** Machine-readable JSON records including UTC timestamps, severity levels, contextual run IDs, project IDs, and exception stack traces. Rotates automatically at 10 MB with 5 backups.

9. Pipeline Health & Staleness Monitoring (`/api/etl/health`)

Two endpoints:

- **GET `/api/etl/health`** (public, for monitors such as UptimeRobot or a load balancer): returns only `{"status": "..."}` , with HTTP **503** when **unhealthy**. The result is cached for 30 seconds, so polling it costs at most one database connection per window. It never returns counts, timings or error text.
- **GET `/api/etl/health/details`** (admin login required; used by the ETL panel): database ping latency, staleness, the last finished run, pending approval counts and row counts.

Freshness is measured from the last run that finished **ok** or **skipped**: a skipped run is a live pipeline that found no source change, which is the normal state between surveys. The threshold is `ETL_MAX_STALENESS_HOURS` (default 24h); with scheduling off, runs happen only on demand and the status turns **degraded** after that window.

Classifications:

- **healthy**: the latest finished run did not fail, and the last **ok/skipped** run is within the threshold.
- **degraded**: the last **ok/skipped** run is older than the threshold.
- **unhealthy**: the approval database is unreachable, the latest finished run failed, or no run has ever succeeded.
- **unknown**: no run has ever finished.

10. Execution Metrics Dashboard & Visual Trend Charts

The Web UI ETL Pipeline Panel (**Runs & Health** tab) provides real-time visual observability over execution performance:

- **KPI Summary Strip**:
- **Avg Duration**: Mean execution latency across runs.
- **Success Rate**: Percentage of successful vs failed runs.
- **Total Merged**: Cumulative rows inserted/updated into **public.anomalies**.
- **Total Runs**: Evaluated run window with skipped count.
- **Interactive SVG Charts**:
- **Duration Trend (Latency)**: Chronological run durations (seconds) color-coded by status (Green = OK, Amber = Skipped, Red = Failed) with interactive hover tooltips.
- **Ingestion Volume & Staged Changes**: Dual bars displaying anomalies written vs. attribute changes held in staging.
- **Outcome Distribution**: Segmented progress bar illustrating proportion of Successful, Skipped, and Failed executions.
- **Run Filtering**: Filter historical runs dynamically by **All**, **Successful**, **Skipped**, or **Failed** to isolate and diagnose anomalies or execution errors.

11. Pipeline Orchestrator & Scheduling Architecture

The pipeline can be scheduled using multiple deployment strategies depending on infrastructure requirements:

Option A: Built-in Structured Cron Orchestrator (Recommended for Containers)

The runner includes a native 5-part cron evaluator with tick alignment (firing precisely on clock boundaries without drift) and graceful signal handling (**SIGTERM/SIGINT**):

```
# Run on standard cron schedule (e.g., every 15 minutes)
python runner/run.py cron --schedule "*/15 * * * *"

# Or via Docker Compose environment variable:
ETL_CRON_SCHEDULE="*/15 * * * *"
```

Option B: Linux Host Crontab / Systemd Timer

Run isolated ephemeral containers on host cron without running continuous daemon containers:

```
# /etc/cron.d/etl-pipeline: run every hour at minute 0
0 * * * * root cd /opt/feedbackapp && docker compose run --rm etl run >> /var/log/etl.log 2>&1
```

Not supported: triggering runs from the database (pg_cron)

An earlier version of this guide showed **pg_cron** calling an HTTP endpoint. That does not work: **POST /api/etl/run** requires a logged-in admin session, the app has no API tokens, and the database container has neither **pg_cron** nor **pg_net**. Use Option A or B.

12. Environment Variables Reference

Variable	Default	Purpose
ETL_CRON_SCHEDULE	<i>(unset)</i>	5-field cron expression for wall-clock aligned container scheduling. Takes precedence over ETL_INTERVAL_SECONDS . Neither set = scheduling off.
ETL_INTERVAL_SECONDS	0 (off)	Fixed sleep interval in seconds, used when ETL_CRON_SCHEDULE is unset.
ETL_WEBHOOK_URL	<i>(unset)</i>	Webhook endpoint for automated Teams / Slack / Discord alert notifications.
ETL_MAX_STALENESS_HOURS	24	SLA threshold in hours before /api/etl/health flags the pipeline as stale.
ETL_DB_HOST	localhost	PostgreSQL server hostname or container service name.
ETL_DB_PORT	5432	PostgreSQL listening port.
ETL_DB_NAME	nolte_geoservices	Target database name.
ETL_DB_USER	etl_pipeline	Least-privilege pipeline role credentials (cannot modify feedback or UPDATE targets directly).
ETL_DB_PASSWORD	<i>(required)</i>	Password for the etl_pipeline role.
ETL_APPROVER_USER	etl_approver	Least-privilege approver role credentials used by the review UI.
ETL_APPROVER_PASSWORD	<i>(required)</i>	Password for the etl_approver role.